

How a Spark Job Runs — End to End

What happens after a data engineer hits "submit," in plain steps

The example: ShopFast has **2 TB of orders** from yesterday in S3. The engineer runs a job to compute **revenue per region** (keep completed orders, join region names, add up amounts). Here's exactly what happens inside the cluster.

The players (who does what)

Driver — the boss. Reads your code, makes the plan, hands out work, tracks progress.

Cluster Manager — the staffing office. Gives you machines (executors).

Executors — the workers. Do the actual processing, in parallel.

Partition — one ~128 MB chunk of the data. The unit of work.

Task — one worker processing one partition.

DAG — the plan — the ordered steps Spark builds before running anything.

The flow, step by step

1 You submit the job
You hand your code (the "recipe") to the cluster with `spark-submit` (or Glue/Databricks does it for you).

2 The Driver starts
The boss shows up, creates the `SparkSession`, and reads your code.

3 Driver asks for workers
The Driver tells the **Cluster Manager** "I need 10 executors." The manager launches them on machines; they report back.

4 Driver builds a smart plan (nothing runs yet)
Spark is **lazy** — it looks at your whole recipe first and optimizes it (e.g., throw away cancelled orders *before* the join). This plan is the **DAG**.

5 An "action" presses go
Writing the result is an **action** — that's what actually triggers the work. Until then, Spark was only planning.

6

The plan is split into stages

Spark cuts the job into **stages** wherever workers must pass data to each other. Stage 1 = scan + filter + join **narrow**. Stage 2 = add up per region **wide**.

7

Work is split into tasks (one per chunk)

2 TB ÷ 128 MB ≈ **16,000 partitions** → 16,000 **tasks**. With 10 executors × 4 cores = **40 running at once**, they flow through in waves.

8

Stage 1 runs (fast, no data moved)

Each worker reads its 128 MB chunk, drops cancelled orders, and tags the region (the small regions table was copied to every worker — a **broadcast** — so no big data movement).

9

The shuffle — the slow part

Now workers **pass data around the network** so all the "Texas" rows end up together, all "California" together, etc. This exchange between stages is the expensive step.

10

Stage 2 runs (add it up)

Each worker sums the amounts for its regions.

11

Result is saved

Workers write the small result (one row per region) back to S3. The Driver reports **success**, and the next pipeline step (catalog → warehouse → dashboard) kicks off.

Where Scala / the JVM fit

Spark's engine is written in **Scala** and runs on the **JVM**. **PySpark is a thin Python layer** — your Python only *builds the plan* and passes it to the JVM (via `Py4J`), so DataFrame/SQL operations run at full JVM speed. The one exception is **Python UDFs**: those send each row from the JVM to a Python process and back, which is why UDFs are slow — prefer built-in functions.

The numbers (this job)

2 TB

input data

~16,000

partitions / tasks (128 MB each)

40

tasks running at once (10 × 4 cores)

1 shuffle

at the "add up per region" step

Two things that make or break it: (1) **broadcast** the small regions table — forget it and Spark shuffles the full 2 TB, turning minutes into hours. (2) Watch for **skew** — if one region has most of the orders, one worker lags and everyone waits.

In one line: submit → the Driver gets workers from the Cluster Manager → builds a lazy plan (DAG) → an action triggers it → the plan splits into stages at the shuffle → each chunk becomes a task run in parallel by executors → results are combined and saved.